

Note Dettagliate sul Calcolatore di Espressioni Matematiche

Panoramica Generale

Questo progetto implementa un **valutatore di espressioni matematiche** che:

- Tokenizza stringhe di input in token semanticamente significativi
- Elabora operatori binari (+, -, *, /, ^) con corrette priorità
- Supporta funzioni matematiche (sin, cos, tan, sqrt, log, exp, etc.)
- Gestisce variabili e assegnamenti
- Utilizza l'algoritmo **Shunting Yard** per convertire notazione infissa a postfissa

FILE 1: Token.cs

Funzione Generale

Definisce la gerarchia di classi che rappresentano i diversi tipi di token che l'espressione può contenere.

Classi e Funzionalità

class Token

- Classe base astratta per tutti i token
- Serve come tipo comune per tutte le sottoclassi

class TNumber : Token

```
Classe base per numeri (letterali e variabili)
Proprietà:
- Value (double): valore numerico del token
```

class TLiteral : TNumber

```
Rappresenta un numero letterale (es. 3.14, 42)
Proprietà:
- mValue (double): il valore effettivo memorizzato
- Value (override): restituisce mValue
Metodi:
- ToString(): restituisce "Literal: {valore}"
```

Funzioni utili:

`double.Parse()` - converte stringa in numero

class TVariable : TNumber

Rappresenta una variabile (es. x, y, z)

Proprietà:

- `mName (string)`: nome della variabile
- `mEval (Evaluator)`: riferimento all'evaluator per recuperare valori
- `Name (readonly)`: accesso pubblico al nome
- `Value (override)`: recupera il valore dalla tabella di simboli

Metodi:

- `ToString()`: restituisce "Variable: {nome}"

Funzioni utili:

`GetVariable()` - richiama `evaluator.GetVariable(Name)`

class TOperator : Token

Classe base per gli operatori

Proprietà:

- `Priority (virtual int)`: priorità dell'operatore (+ override specifici)
- `FinalPriority (int)`: priorità considerando le parentesi

class TOpBinary : TOperator

Rappresenta operatori binari (+, -, *, /, ^)

Proprietà:

- `mOp (char)`: il carattere dell'operatore
- `Op (readonly)`: accesso pubblico
- `Priority (override)`: restituisce priorità
 - +, - 'priorità 1
 - *, / 'priorità 2
 - ^ 'priorità 3

Metodi:

- `Apply(double a, double b)`: esegue l'operazione
 - '+' 'a + b
 - '-' 'a - b
 - '*' 'a * b
 - '/' 'a / b
 - '^' 'Math.Pow(a, b) [elevamento a potenza]
- `ToString()`: restituisce "Binary: {operatore}"

Funzioni utili:

Math.Pow(a, b) - calcola a^b

class TopFunction : TOperator

Rappresenta funzioni matematiche (sin, cos, tan, sqrt, log, exp, asin, acos, atan)

Proprietà:

- mFunc (string): nome della funzione
- Func (readonly): accesso pubblico
- Priority (override): sempre 4 (priorità massima)

Metodi:

- Apply(double f): esegue la funzione
- sin/cos/tan: angolo in gradi 'conversione a radianti' calcolo
- asin/acos/atan: risultato in radianti 'conversione a gradi
- sqrt: Math.Sqrt(f)
- log: Math.Log(f) [logaritmo naturale]
- exp: Math.Exp(f) [e^f]
- D2R(double f): gradi 'radianti' ($f * \frac{\pi}{180}$)
- R2D(double f): radianti 'gradi' ($f * \frac{180}{\pi}$)
- ToString(): restituisce "Function: {nome}"

Funzioni utili:

Math.Sin/Cos/Tan(radianti)
Math.Asin/Acos/Atan(valore)
Math.Sqrt(f)
Math.Log(f)
Math.Exp(f)
Math.PI - costante π

class TPunctuation : Token

Rappresenta punteggiatura (parentesi)

Proprietà:

- mPunct (char): il carattere ('(' o ')')
- Punct (readonly): accesso pubblico

Metodi:

- ToString(): restituisce "Punctuation: {carattere}"

Nota: usata per gestire priorità delle operazioni

class TEnd : Token

Marker che indica la fine dei token
Nessuna proprietà aggiuntiva
Usato come segnale di terminazione nel tokenizer

class TError : Token

Rappresenta un token di errore
Proprietà:
- mMessage (string): descrizione dell'errore
- Message (readonly): accesso pubblico
Metodi:
- ToString(): restituisce "Error: {messaggio}"

FILE 2: Tokenizer.cs

Funzione Generale

Converte una stringa di input in una sequenza di token. Implementa l'**analisi lessicale** (lexical analysis).

Proprietà della Classe

- mText (string): testo da tokenizzare
- mEval (Evaluator): riferimento all'evaluator
- mN (int): posizione corrente nel testo
- mFuncs (string[]): lista di nomi di funzioni riconosciute

Metodi Principali

Token GetNext()

Legge il prossimo token dalla stringa di input
Algoritmo:
1. Salta gli spazi
2. Legge il primo carattere:
- Se operatore (+, -, *, /, ^, =) 'crea TOpBinary
- Se cifra (0-9) 'chiama GetLiteral()
- Se parentesi () 'crea TPunctuation
- Se lettera (a-z) 'chiama GetIdentifier()
- Altrimenti 'crea TError

3. Se fine stringa 'crea TEnd

Funzioni utili:

char.IsDigit() - verifica se è cifra [usato tramite 'ch is >= '0' and <= '9']

char.IsLetter() - verifica se è lettera [usato tramite 'ch is >= 'a' and <= 'z']

Token GetLiteral()

Estrae un numero decimale dalla stringa

Algoritmo:

1. Registra posizione iniziale
2. Continua finché incontra cifre o punto decimale
3. Estrae substring dal testo
4. Converte a double con CultureInfo invariante

Funzioni utili:

string.Substring(start, length) - estrae porzione di stringa

double.Parse(string, IFormatProvider) - converte stringa a double

System.Globalization.CultureInfo.InvariantCulture - formato numero indipendente da locale

Token GetIdentifier()

Estrae un identificatore (variabile o funzione)

Algoritmo:

1. Registra posizione iniziale
2. Continua mentre incontra lettere o cifre
3. Estrae substring
4. Verifica se è in mFuncs:
 - Sì 'crea TOpFunction
 - No 'crea TVariable

Funzioni utili:

List<T>.Contains(T) - verifica se elemento è nella lista

string.Substring(start, length)

Esempio di Tokenizzazione

Input: "3.14 + sin(x)"

Output:

TLiteral(3.14)

TOpBinary(+)

TOpFunction(sin)

```
TPunctuation()  
TVariable(x)  
TPunctuation()
```

FILE 3: Evaluator.cs

Funzione Generale

Valuta le espressioni matematiche utilizzando l'**algoritmo Shunting Yard** di Dijkstra. Questo algoritmo converte notazione infissa a postfissa ed esegue i calcoli.

Proprietà della Classe

- mOperands (Stack<double>): stack dei numeri da elaborare
- mOperators (Stack<TOperator>): stack degli operatori da applicare
- mVariables (Dictionary<string, double>): tabella di simboli (variabili memorizzate)
- mBasePriority (int): priorità base, incrementata da parentesi

Metodi Principali

double Evaluate(string input)

Metodo principale - valuta un'espressione completa

Algoritmo:

1. Reset() - pulisce gli stack
2. Crea Tokenizer e legge tutti i token fino a TEnd
3. Rileva assegnamenti (var = espressione):
 - Se primo token è TVariable e secondo è '='
 - Salva la variabile e rimuove i token di assegnamento
4. Elabora ogni token con Process()
5. Applica operatori rimanenti con ApplyOperator()
6. Validazioni finali:
 - Parentesi bilanciate (mBasePriority == 0)
 - Nessun operatore rimasto (mOperators.Count == 0)
 - Esattamente un operando (mOperands.Count == 1)
7. Se assegnamento, memorizza variabile
8. Restituisce risultato

Funzioni utili:

Tokenizer.GetNext() - legge token

List<T>.RemoveRange(index, count) - rimuove elementi

Dictionary<K,V>.TryGetValue(key, out value) - legge con controllo

```
Stack<T>.Pop() - rimuove e restituisce elemento  
Math.Round(double, int) - arrotonda a N decimali
```

void Process(Token token)

Elabora un singolo token secondo Shunting Yard
Casi:

CASO 1 - TNumber (TLiteral o TVariable):

' Stack degli operandi: push Value

CASO 2 - TOperator:

' Calcola FinalPriority = Priority + mBasePriority

' Mentre non è ok fare push: ApplyOperator()

' Stack operatori: push operatore

CASO 3 - TPunctuation:

' '(' aumenta mBasePriority di 10

' ')' diminuisce mBasePriority di 10

Funzioni utili:

Stack<T>.Push(T) - aggiunge elemento

Stack<T>.Peek() - legge senza rimuovere

switch expression - pattern matching

void ApplyOperator()

Applica l'operatore in cima allo stack degli operatori

Algoritmo:

1. Pop operatore dallo stack

2. Se TOpBinary:

- Pop due operandi (a, b)
- Calcola b OP a [ordine importante!]
- Push risultato

3. Se TOpFunction:

- Pop un operando
- Applica funzione
- Push risultato

Funzioni utili:

Stack<T>.Pop()

TOpBinary.Apply(a, b)

TOpFunction.Apply(f)

bool OkToPush(TOperator op)

Determina se è sicuro fare push del nuovo operatore
Regole Shunting Yard:
- Se stack operatori vuoto ' TRUE (push subito)
- Se FinalPriority dell'operatore precedente < FinalPriority nuovo ' TRUE
- Altrimenti ' FALSE (applicare operatore precedente prima)
Funzioni utili:
Stack<T>.Count - numero elementi
Stack<T>.Peek()

double GetVariable(string name)

Recupera il valore di una variabile dalla tabella di simboli
Se non esiste ' genera errore EvalException
Funzioni utili:
Dictionary<K,V>.TryGetValue(key, out value) - lettura sicura

void Reset()

Resetta gli stack e la priorità base tra valutazioni
Funzioni utili:
Stack<T>.Clear() - svuota lo stack
Dictionary<K,V>.Clear() - svuota il dizionario

void Error(string s)

Genera un'eccezione EvalException con il messaggio
Supporta il propagarsi degli errori nel parser

Esempio di Esecuzione Shunting Yard

Input: "2 + 3 * 4"
Tokens: [TLiteral(2), TOpBinary(+), TLiteral(3), TOpBinary(*), TLiteral(4)]

Elaborazione:


```
1. TLiteral(2)      'mOperands: [2]
2. TOpBinary(+)     'mOperators: [+]
3. TLiteral(3)      'mOperands: [2, 3]
4. TOpBinary(*)     'Priorità * > + 'mOperators: [+, *]
5. TLiteral(4)      'mOperands: [2, 3, 4]
6. Fine             'Applica * 'mOperands: [2, 12]
  'Applica + 'mOperands: [14]
Output: 14
```

FILE 4: Program.cs

Funzione Generale

Implementa l'interfaccia **REPL** (Read-Eval-Print Loop) - ciclo interattivo con l'utente.

Funzionamento

Ciclo infinito finché l'utente non digita "exit":

1. Prompt (">") visualizzato
2. Legge input da console
3. Converte a minuscole e rimuove spazi
4. Se "exit" 'esce dal loop
5. Altrimenti:
 - Chiama eval.Evaluate(input)
 - Se successo:
 - * Imposta colore blu
 - * Stampa risultato arrotondato a 10 decimali
 - Se errore:
 - * Imposta colore rosso
 - * Stampa messaggio errore
6. Resetta colore console
7. Torna a step 1

Funzioni utili:

```
Console.Write() / Console.WriteLine()
Console.ReadLine() - legge da input
string.Trim() - rimuove spazi all'inizio e fine
string.ToLower() - converte a minuscole
string.Split() - divide stringhe [potrebbe essere usato]
Console.ForegroundColor - imposta colore testo
Console.ResetColor() - resetta colore a default
Console.Clear() - pulisce schermo [non usato ma disponibile]
```

Esempio di Interazione

```
> 2+3
5
> sin(90)
1
> x=10
10
> x*2
20
> exit
[termina il programma]
```

Funzioni Utili C# Utilizzate nel Progetto

String Methods

Funzione	Uso	Esempio
<code>string.Substring(start, len)</code>	Estrae porzione di stringa	<code>"hello".Substring(1, 3)</code> 'ell'
<code>string.Trim()</code>	Rimuove spazi iniziali/finali	<code>" hello ".Trim()</code> 'hello'
<code>string.ToLower()</code>	Converte a minuscole	<code>"HELLO".ToLower()</code> 'hello'
<code>string.IsNullOrEmpty()</code>	Controlla se vuota	<code>IsNullOrEmpty("")</code> 'true'

Type Conversion

Funzione	Uso	Esempio
<code>double.Parse(string, IFormatProvider)</code>	Converte stringa a numero	<code>double.Parse("3.14")</code> '3.14'
<code>CultureInfo.InvariantCulture</code>	Formato numero indipendente da locale	Usato con Parse

Collection Methods

Funzione	Uso	Esempio
<code>List<T>.Contains(T)</code>	Verifica se elemento presente	<code>list.Contains(x)</code> 'bool
<code>List<T>.RemoveRange(index, count)</code>	Rimuove elementi multipli	<code>list.RemoveRange(0, 2)</code>
<code>Stack<T>.Push(T)</code>	Aggiunge elemento in cima	<code>stack.Push(5)</code>
<code>Stack<T>.Pop()</code>	Rimuove e restituisce elemento in cima	<code>var x = stack.Pop()</code>
<code>Stack<T>.Peek()</code>	Legge elemento in cima senza rimuovere	<code>var x = stack.Peek()</code>
<code>Stack<T>.Count</code>	Numero elementi	<code>stack.Count</code> 'int
<code>Stack<T>.Clear()</code>	Svuota stack	<code>stack.Clear()</code>
<code>Dictionary<K,V>.TryGetValue(K, out V)</code>	Lettura sicura con controllo	<code>dict.TryGetValue(key, out val)</code>
<code>Dictionary<K,V>[K]</code>	Accesso con setter	<code>dict[key] = value</code>
<code>Dictionary<K,V>.Clear()</code>	Svuota dizionario	<code>dict.Clear()</code>

Math Functions

Funzione	Uso
<code>Math.Sin(radiani)</code>	Seno
<code>Math.Cos(radiani)</code>	Coseno
<code>Math.Tan(radiani)</code>	Tangente
<code>Math.Asin/Acos/Atan(valore)</code>	Funzioni inverse

<code>Math.Sqrt(x)</code>	Radice quadrata
<code>Math.Log(x)</code>	Logaritmo naturale
<code>Math.Exp(x)</code>	e elevato a x
<code>Math.Pow(a, b)</code>	a elevato a b
<code>Math.PI</code>	Costante π (3.14159...)
<code>Math.Round(value, decimals)</code>	Arrotonda a N decimali

Console Methods

Funzione	Uso
<code>Console.Write(object)</code>	Stampa senza newline
<code>Console.WriteLine(object)</code>	Stampa con newline
<code>Console.ReadLine()</code>	Legge riga da input
<code>Console.ForegroundColor</code>	Imposta colore testo (get/set)
<code>Console.ResetColor()</code>	Resetta colore default

Language Features

Funzione	Uso	Esempio
<code>switch expression</code>	Pattern matching compatto	<code>x switch { 1 => "one", _ => "other" }</code>
<code>is</code> pattern	Verifica tipo e decomposizione	<code>if (token is TVariable tv)</code>
<code>??</code> operator	Coalescing null	<code>a ?? b</code> (usa b se a è null)
<code>foreach</code> loop	Itera su collezione	<code>foreach(var x in list)</code>

try-catch	Gestione eccezioni	try { } catch (Exception e) { }
-----------	--------------------	---------------------------------

Custom Exception

Classe	Uso
EvalException : Exception	Eccezione personalizzata per errori di valutazione

Diagramma del Flusso

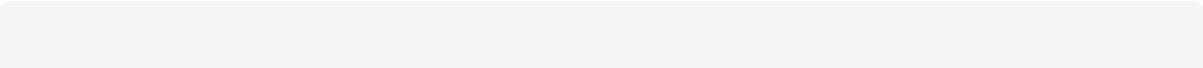
```
Input: "2 + 3 * 4"
"
[Program.cs: Main()]
"
[Evaluator.Evaluate(string)]
"
[Tokenizer.GetNext()] ' Produce token
"
while (!TEnd) {
[Token.cs] ' Crea TLiteral, TOpBinary, etc.
}
"
[Evaluator.Process(token)] ' Algoritmo Shunting Yard
"
[Evaluator.ApplyOperator()] ' Esegue calcoli
"
Output: 14
"
[Program.cs] ' Stampa in blu con Console.WriteLine()
```

Concetti Chiave

1. Shunting Yard Algorithm (Dijkstra)

Converte espressioni da notazione infissa (2+3) a postfissa (2 3 +) e le valuta mantenendo priorità degli operatori.

2. Pattern Matching (C# 8+)



```
switch (token) {  
  case TNumber num: ...  
  case TOperator op: ...  
}
```

3. Type Casting e Inheritance

Sfrutta ereditarietà per polimorfismo:

- TNumber ' TLiteral, TVariable
- TOperator ' TOpBinary, TOpFunction

4. Stack-based Evaluation

Usa due stack (operandi e operatori) per valutare espressioni in modo efficiente e gestire priorità.

5. Variable Storage

Tabella di simboli (Dictionary) memorizza variabili tra valutazioni, permettendo assegnamenti come `x=10` e successivo uso di `x`.

Casi di Uso Supportati

```
Operazioni aritmetiche: 2+3, 5*6, 10/2, 3^2  
Funzioni trigonometriche: sin(90), cos(0), tan(45)  
Funzioni inverse: asin(1), acos(0), atan(1)  
Funzioni esponenziali: sqrt(16), log(2.718), exp(1)  
Variabili: x=5, y=x+3  
Parentesi: (2+3)*4  
Combinazioni: sin(90)*(x+2)
```